Adaptive Progressive Retrieval Attention: Confidence-Guided Routing, Monotonic Retry, and a Bounded Memory Runtime

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 16, 2026

Abstract

Progressive Retrieval Attention (PRA) separates conceptual memory discovery from physical native-key/value (K/V) activation. Earlier experiments used fixed routing and materialization settings. This paper asks whether one runtime can spend little on easy queries, escalate difficult queries, reuse prior work, and stop under explicit confidence and resource bounds.

We introduce a discrete controller over three validation-frozen effort profiles. Each profile instantiates query facets, retained roots, neighbors, hop depth, conceptual and physical budgets, thresholds, search/consumer layers, granularity, and disclosure policy. The runtime exposes manual and automatic modes, a monotonic retry state machine, oracle-free features, calibrated stop signals, structured traces, exact batched native-Q/K search, interval gathering, a page-table prototype, cache policies, and heterogeneous batching.

The adaptive study uses 95 validation and 65 held-out example–seed units from frozen Paper-2.5 routing traces. On QASPER, the learned controller matches the always-high chain-completion rate (.900) while reducing mean abstract effort from 152.0 to 60.8, a 60.0% reduction. On HotpotQA, both learned and always-high control reach only .371; the controller assigns the full effort because 31 of 35 held-out units require the highest profile under the oracle minimum-effort audit. This is appropriate allocation, not a compute saving. A cheap-first retry reaches the same quality, but costs 156.0 on HotpotQA and 141.9 on QASPER because repeated generation offsets reuse. Combined routing features predict failure with AUROC .824 and AUPRC .853. In a separate controlled output trace, entropy alone is weak (AUROC .403), while answer margin and combined output signals separate errors; entropy is therefore not a sufficient stop rule.

Prototype systems measurements show that batched GEMM is 2,527× faster than a Python per-key loop at 4,096 keys. The present coarse index is slower than GEMM at this scale and reaches only .648 Top-8 recall with eight probes. A fused `index_select` gather is parity-correct but has no consistent CPU advantage at these sizes, whereas Python page lookup remains 14–37× slower than contiguous gathering. Controlled matched-budget comparisons include iterative RAG, native context, and mechanism-faithful K/V selector proxies; they are not production baseline implementations. The evidence supports adaptive memory allocation and a reproducible runtime contract, but not yet a production vLLM speed claim.

1 Introduction

A fixed memory policy is wrong in two directions. A narrow policy wastes quality on questions whose evidence is dispersed or connected by several relations. A broad policy wastes search, transfer, K/V capacity, and attention competition on questions that can be answered from one compact

region. The relevant systems question is therefore not only which memory should be retrieved, but how much effort the current query justifies.

PRA provides several independent control surfaces. A runtime may form one or several query facets, retain one or several roots, expand different numbers of neighbors and hops, search at different layers, and materialize anything from a compact evidence interval to a broader source region. Paper 3 showed that conceptual breadth and physical disclosure are not the same budget: once identity is fixed, exact native-K/V evidence can outperform a whole parent while using far fewer physical states. The present work turns those controls into an adaptive inference contract.

The central test is

$$Q_{\text{adaptive}} \approx Q_{\text{always-high}} \quad \text{while} \quad \mathbb{E}[C_{\text{adaptive}}] < C_{\text{always-high}}, \tag{1}$$

where Q is held-out output or retrieval quality and C includes conceptual search, physical K/V, generation retries, and measured latency. Equation 1 is not guaranteed on every dataset. If almost every query is hard, the correct controller should spend the high budget.

This paper makes six contributions:

1. It defines a public PRA control vector and freezes three monotonic effort profiles rather than predicting unconstrained continuous parameters.
2. It implements hand-rule and learned minimum-effort controllers using only observable query, routing, output, and memory features; evaluator labels are rejected by the serving API.
3. It provides manual and automatic retry modes with incremental state reuse, hard budgets, semantic answer consistency, and structured decision traces.
4. It adds exact batched and coarse-to-fine native-Q/K search, fused interval gathering, a fixed-page K/V prototype, cache/prefetch policies, and heterogeneous batching.
5. It evaluates validation-frozen adaptive compute, calibration, retry recovery, and oracle minimum effort on held-out HotpotQA and QASPER traces.
6. It constructs matched-budget RAG, long-context, and K/V-selection frontiers while clearly separating controlled proxies from production third-party implementations.

Full-weight PRA retraining is deliberately absent. That intervention belongs to Paper 4. Here the scientific object is adaptive inference over frozen routing and consumer behavior.

2 Adaptive PRA as a Control Problem

2.1 Control vector

At attempt t, the runtime selects

$$a_t = (F, R, K, H, B_c, B_{kv}, \theta, L_s, L_c, G, M), \tag{2}$$

where F is the facet policy, R retained roots, K candidate neighbors per expansion, H hop depth, B_c conceptual parent budget, B_{kv} physical native-K/V budget, θ a closure threshold, L_s search layers, L_c consumer layers, G routing/materialization granularity, and M disclosure policy. We separate B_c and B_{kv} because Paper 3 found that broad search can remain compatible with compact evidence-only activation.

The first public implementation uses three profiles (Table 1). They are not universal hyperparameters; they are a stable SDK contract and an initial experimental ladder.

Table 1: Validation-frozen effort profiles. The layer identifiers refer to the inherited 28-layer Qwen experiments. Lower routing thresholds permit broader closure.

Profile	facets	R	K	H	B_c	B_{kv}	G	consumer layers
E0 low	1	1	2	0	2	256	256	27
E1 medium	2	2	4	1	4	512	128	26–27
E2 high	4	4	8	3	8	1024	64	24–27

Escalation is monotonic in permitted facets, roots, neighbors, hops, conceptual budget, K/V budget, and layer sets. A lower θ is broader. Code rejects a ladder in which a later profile removes a prior resource. Monotonic permission does not imply monotonic answer quality: extra memory may dilute attention, so retries must also record correct-to-incorrect transitions.

2.2 Observable state and leakage boundary

The controller may observe cheap query statistics, score gaps, routing entropy, root competition, facet agreement, frontier shape, newly discovered memory, path convergence, output entropy, answer log probability and margin, retry consistency, active K/V, memory-attention ratio, and a relevance-density proxy. Ground-truth answers, oracle identities, gold evidence, and evidence recall are evaluator fields. They cannot enter the serving feature constructor.

For vocabulary distribution p_t, token entropy is

$$H_t = -\sum_v p_t(v) \log p_t(v), \qquad \bar{H} = \frac{1}{T}\sum_{t=1}^{T} H_t. \tag{3}$$

Entropy is one signal. A fluent wrong answer can have low entropy. We therefore compare entropy-only, answer-margin, routing-only, and combined predictors rather than defining confidence as a single scalar by fiat.

2.3 Minimum-effort prediction

For validation example i, let

$$e_i^* = \min\{e \in \{E0, E1, E2\} : Q_i(e) \geq q_i^*\}, \tag{4}$$

where q_i^* is the target quality criterion. If no profile succeeds, $E2$ is retained as the bounded fallback. The oracle e_i^* is used only to train or analyze the controller. At test time, a standardized ridge classifier predicts one profile from the initial observable feature vector. The classifier is deliberately small, deterministic, and serializable without a new runtime dependency.

Controller regret is

$$\text{regret}_i = C(\hat{e}_i) - C(e_i^*) \tag{5}$$

subject to reporting whether $Q_i(\hat{e}_i)$ matches $Q_i(e_i^*)$. Negative regret without matched quality is not a saving.

3 Retry and Runtime Design

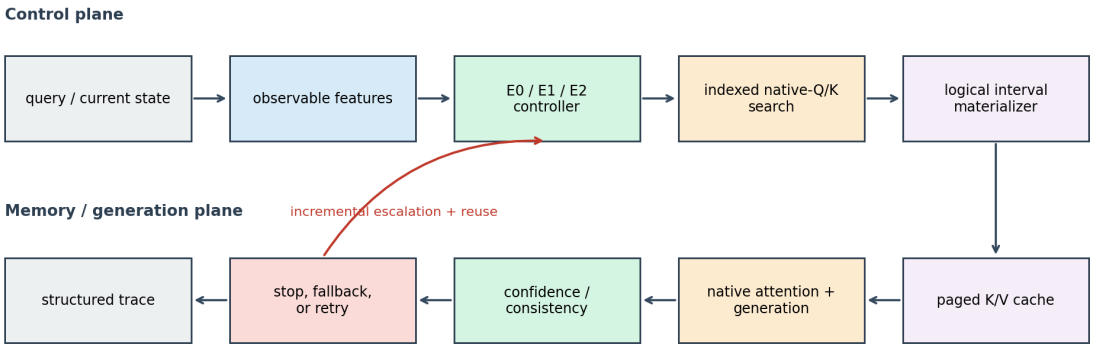

Figure 1: Adaptive PRA runtime. The control plane chooses a bounded profile from observable state. Search and K/V state are reused when confidence, consistency, or closure signals request a broader attempt. The trace exposes decisions and metrics, not hidden reasoning.

3.1 State machine

Attempt zero uses the selected profile. A manual caller may name low, medium, or high effort and explicitly request one broader retry. Automatic mode applies a calibrated stop rule. If the rule fails, the next profile may widen selected controls while preserving prior roots, search results, K/V pages, and prefix state. Search budget, active K/V, retry count, and latency are hard caps.

The public transition is:

```
profile = controller.choose(observable_features)
while True:
    result = execute(profile, reusable_state=previous)
    stop, signals = stop_policy(result.features, result.error_probability)
    trace.append(public_decision(result, profile, signals))
    if stop or budget_exhausted(profile, trace):
        return result, trace
    profile = next_monotonic_profile(profile)
    previous = result
```

Answers across attempts are compared with normalized token F1. This is a cheap semantic proxy, not a full entailment judge. The runtime records initially-wrong-to-correct, initially-correct-to-wrong, and no-change retries separately.

3.2 Structured traces

A trace contains attempt index, profile and control vector, active K/V, selected-parent count, incorrect-answer probability, routing entropy, answer margin, consistency, search/materialization/generation time, reuse counts, escalation signals, and stop reason. It does not contain hidden chain-of-thought. Evaluator outcomes live in experiment rows outside the controller feature path.

3.3 Indexed native-Q/K search

Let $Q \in \mathbb{R}^{n_q \times d}$ be native query states and $K_m \in \mathbb{R}^{n_m \times d}$ indexed memory keys. Exact batched search computes

$$S = QK_m^\top, \qquad I = \mathrm{TopK}(S, k). \tag{6}$$

This removes Python per-key scoring and dense all-pairs graph construction. The prototype also uses deterministic coarse centroids: select p clusters, gather their candidates, then apply exact Top-K inside that subset. Approximation is acceptable only when recall/latency beats exact GEMM at the deployment scale.

3.4 Materialization and page layout

The materializer validates source-relative half-open intervals, merges overlap and adjacency, builds one token index, and applies `index_select` to K and V with shape $[B, H, T, D_h]$. A fixed-page store maps $(\text{request}, \text{page})$ to $[H, P, D_h]$ tensors and supports LRU, LFU, and hybrid eviction. This establishes page accounting, fragmentation, and exact gather parity. It is not a vLLM kernel. PagedAttention controls where K/V is stored [5]; PRA controls which logical memory becomes active. A production integration must join those decisions without replaying selected tokens through Python.

4 Experimental Protocol

4.1 Adaptive-compute cohort

The controller study reuses frozen Paper-2.5 semantic exploration traces. The selected root policy is seed agreement at .6; transition policy is fixed Top-4. Fractions .1, .2, and .4 instantiate E0, E1, and E2. There are 95 validation and 65 held-out example–seed units across HotpotQA and QASPER. Validation chooses the hand rule, learned classifier, failure predictor, and retry threshold. No threshold is changed after held-out evaluation.

The quality endpoint is complete evidence-chain recovery. This is a routing endpoint, not generated answer accuracy. Output confidence is therefore evaluated separately with Paper 3's controlled model, using its validation/held-out split and actual prediction entropy, NLL, answer margin, and correctness. Keeping the two cohorts separate avoids fabricating a joint output/routing signal that was not measured.

Abstract effort is deterministic:

$$C(e) = RK(H + 1) + B_c + B_{kv}/64. \tag{7}$$

A retry pays the final monotonic profile plus a fixed controlled regeneration charge of two units per additional attempt. Measured routing latency is reported separately.

4.2 Systems protocol

Search, gather, page, cache, and batching benchmarks run on CPU with PyTorch 2.12.1. Exact search uses 32 queries, 64-dimensional keys, Top-8, and 256–4,096 memories. Kernel tests use 8 heads and head dimension 32. Page tests store 1,537 tokens and select 128 clustered or dispersed tokens. Batch tests use 1–32 heterogeneous requests. We report medians and mark HBM estimates as extrapolated.

Held-out Qwen measurements are inherited from the Paper-2.5 Gate-3 test artifact and normalized into one serving schema. They include TTFT, TPOT, total generation time, routing, materialization, transfer, active K/V, and peak GPU allocation. They are single-request observations, not an adaptive end-to-end serving run.

4.3 Baseline discipline

The matched synthetic corpus contains 1,024 documents and 200 two-hop queries. The query retrieves a bridge; the answer evidence is linked from that bridge. Native context, truncation, one-shot RAG, multi-query RAG, iterative RAG, fixed PRA, and adaptive PRA share the corpus and retrieval geometry. This controlled construction tests cost accounting and the need for iteration. It does not replace a full production RAG evaluation [6, 13].

H2O [17], SnapKV [7], StreamingLLM [15], and ClusterKV [8] are represented by mechanism-faithful controlled selectors at matched active-K/V budgets. No upstream kernel is claimed. Sparse architectural methods are discussed rather than experimentally mixed with incompatible checkpoints.

5 Adaptive-Compute Results

5.1 Oracle ceiling and direct control

The oracle audit is strongly dataset-dependent. On HotpotQA, 31 of 35 held-out units require E2, two require E1, and two require E0. On QASPER, 24 of 30 require only E0 and six require E2. This heterogeneity creates a large QASPER adaptive-compute opportunity but almost none on HotpotQA.

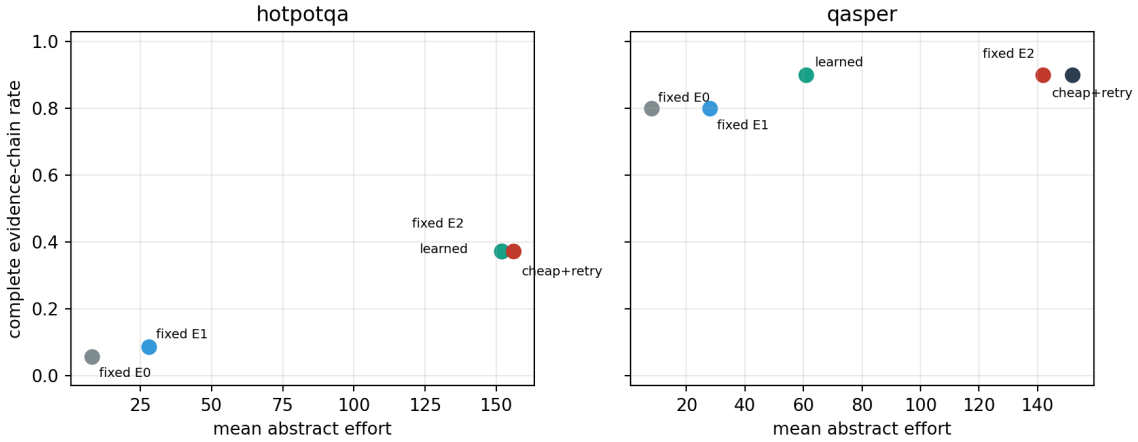

Figure 2: Held-out complete-chain quality versus mean abstract effort. Learned control matches E2 on both datasets. It saves effort on QASPER but not on the predominantly hard HotpotQA cohort.

Table 2: Held-out adaptive-compute frontier. Cost includes conceptual search and physical K/V; retry rows add the controlled regeneration charge.

Dataset	Policy	chain quality	mean cost	attempts	escalation
HotpotQA	fixed E0	.057	8.0	1.00	.000
HotpotQA	fixed E2	.371	152.0	1.00	.000
HotpotQA	learned direct	.371	152.0	1.00	.000
HotpotQA	cheap + retry	.371	156.0	3.00	1.000
QASPER	fixed E0	.800	8.0	1.00	.000
QASPER	fixed E2	.900	152.0	1.00	.000
QASPER	learned direct	.900	60.8	1.00	.000
QASPER	cheap + retry	.900	141.9	2.83	.933

The learned direct policy reduces QASPER mean effort by 60.0% with no chain-quality loss. Its mean oracle cost regret is 24 units and quality match is 1.0, so further gains remain possible. On HotpotQA, the learned and hand controllers both match oracle-profile quality but incur positive regret because a few easy cases are conservatively sent to E2.

5.2 Retry recovery is not free

Cheap-first retry corrects 31.4% of HotpotQA units that fail E0 and 10.0% of QASPER units. It does not break an initially correct unit in this trace. However, HotpotQA reaches E2 on every request and pays two extra generation attempts, raising cost from 152.0 to 156.0. QASPER stops early on two units but still escalates 93.3%, reducing cost only to 141.9. Retry is valuable as a recovery and fallback mechanism; the learned initial profile is the main source of QASPER savings.

This result also distinguishes routing reuse from generation reuse. Roots and K/V states can be preserved incrementally, but a changed answer may still require regeneration. A serving claim must measure discarded tokens and prefix-cache reuse rather than treating monotonic search as free.

5.3 Confidence and selective risk

Table 3: Held-out incorrect/failure prediction. Output rows use the controlled Paper-3 model; routing rows use frozen Paper-2.5 traces.

Cohort	Signals	AUROC	AUPRC	ECE	Brier
Output	entropy only	.403	.899	.229	.145
Output	answer margin only	1.000	1.000	.277	.133
Output	combined output	1.000	1.000	.291	.117
Routing	routing entropy only	.652	.690	.043	.249
Routing	root gaps	.802	.816	.117	.197
Routing	combined runtime	.824	.853	.139	.187

The output cohort is small (40 held-out observations, 90% incorrect), so perfect rank separation for answer margin is diagnostic rather than a deployment estimate. Entropy-only ranking is worse than random in this cohort despite high AUPRC caused by class imbalance. No incorrect output is assigned less than .2 error probability by the fitted predictors, but this does not establish that confidently wrong answers are absent in broader language tasks.

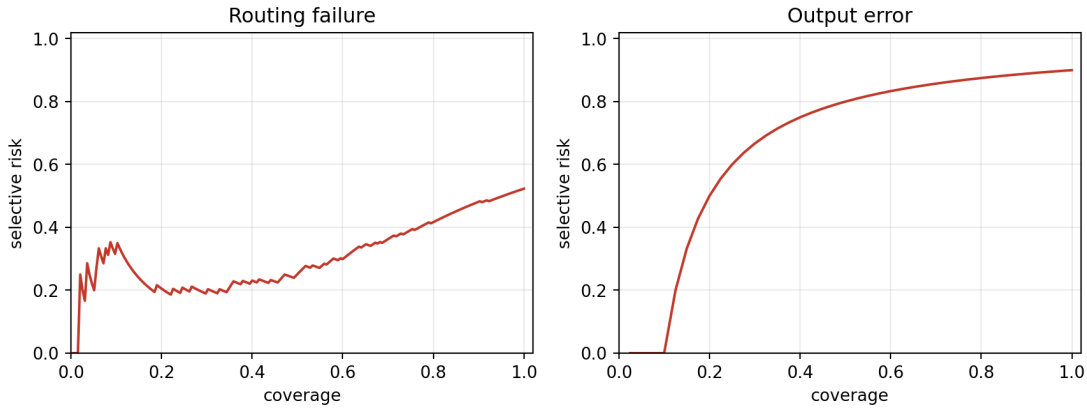

Figure 3: Selective risk as increasingly uncertain observations are accepted. Curves are reported separately because routing and output confidence were not observed on one unified live cohort.

6 Systems Results

6.1 Exact GEMM is the current small-index default

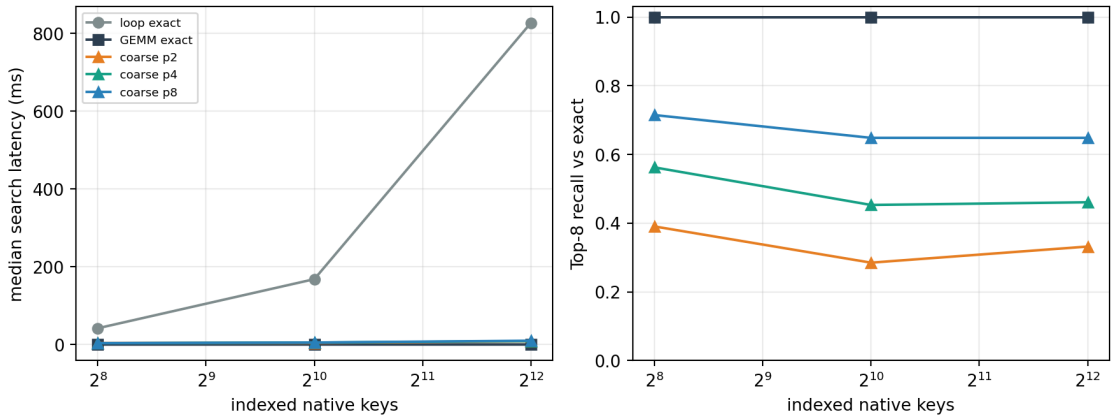

Figure 4: Native-Q/K search. Exact GEMM removes Python per-key overhead. The present deterministic coarse index loses recall and does not yet beat GEMM at these memory sizes.

At 4,096 keys, the Python loop takes .827 seconds for 32 queries while exact GEMM takes .327 ms, a 2,527× speedup with identical Top-8 results. Coarse-to-fine search with two, four, and eight probes reaches .332, .461, and .648 recall relative to exact, respectively, while taking 8–13 ms. The ANN path is therefore not adopted as a default. It requires a mature FAISS/HNSW/GPU index and larger-scale crossover measurements before it can support a speed claim.

6.2 Gather and page layout

Table 4: Median standalone CPU gather time in milliseconds. Every row has exact K/V parity.

Source tokens	Python slice/cat	fused index-select
512	.197	.182
2,048	.195	.227
8,192	.191	.198

The one-index path has exact parity but no consistent CPU speed advantage in this small benchmark; it is not a fused CUDA kernel. The fixed-page Python lookup is substantially slower: depending on page size and selection dispersion, it costs 14–37× contiguous `index_select`. Pages of 16–128 tokens waste 15–127 tail slots for the 1,537-token request. Clustered selection touches 9, 5, 3, and 2 pages; dispersed selection touches 97, 49, 25, and 13. These numbers explain what a production page gather should optimize, not how fast that gather will be.

6.3 Cache and prefetch

Without prefetch, the best controlled hit rate is .921 at 32 pages; LRU, LFU, and hybrid tie because the full working set nearly fits. At 16 pages LFU reaches .896, above LRU's .833, because the workload contains a stable hot set plus a moving path. Graph-neighbor prefetch is measured, while the next-access trace is reported only as an oracle upper bound and never as a deployable predictor. Useful and wasted prefetch transfers are explicit artifact columns.

6.4 Heterogeneous batching

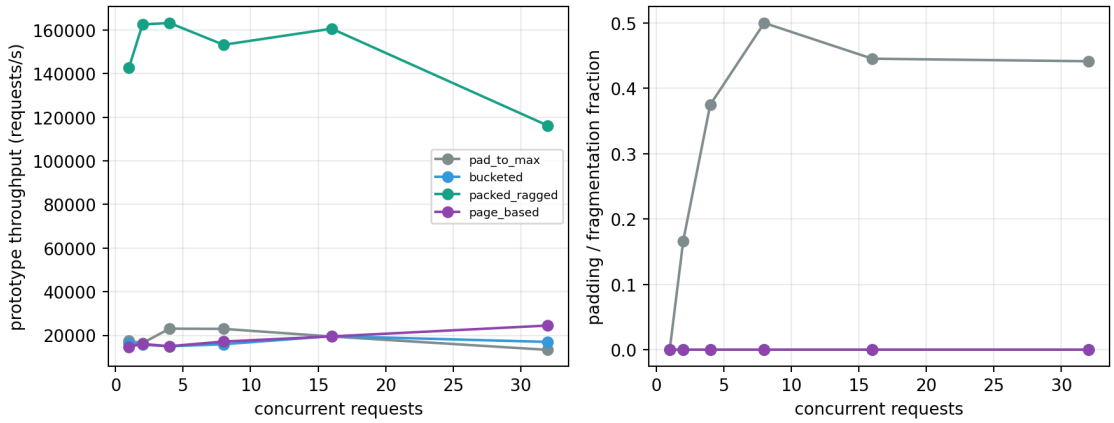

Figure 5: Controlled CPU batching. Packed ragged execution avoids padding and is fastest for this small per-request matrix-vector workload; this is not a GPU attention-kernel benchmark.

At concurrency 32, pad-to-maximum wastes 44.1% of allocated token slots. Bucketed, page-based, and packed layouts have zero waste for the chosen lengths. Packed per-request operations reach 128,154 requests/s versus 13,273 for pad-to-maximum in this CPU microbenchmark. On GPU, launch overhead and fused kernels can reverse this ordering; the artifact therefore records device and measurement scope.

6.5 Inherited Qwen serving observations

Table 5: Mean held-out single-request Qwen observations inherited from Gate 3. Active K/V counts layer-token states, not unique source tokens.

Dataset	Effort	active K/V states	TTFT (s)	total (s)
2Wiki	E0	2,200	.169	1.000
2Wiki	E1	4,267	.190	1.348
2Wiki	E2	6,215	.198	1.465
MuSiQue	E0	7,562	.320	3.740
MuSiQue	E1	44,683	.705	7.688
MuSiQue	E2	47,869	.737	7.936

These rows demonstrate that effort can materially change K/V and latency. They do not demonstrate that the new controller produces the same values end to end, because the adaptive controller was evaluated on frozen routing traces rather than a live Qwen scheduler. GPU utilization and dollar cost were not captured on the local hardware and remain explicitly missing.

7 Matched Baselines

7.1 RAG, iterative retrieval, and long context

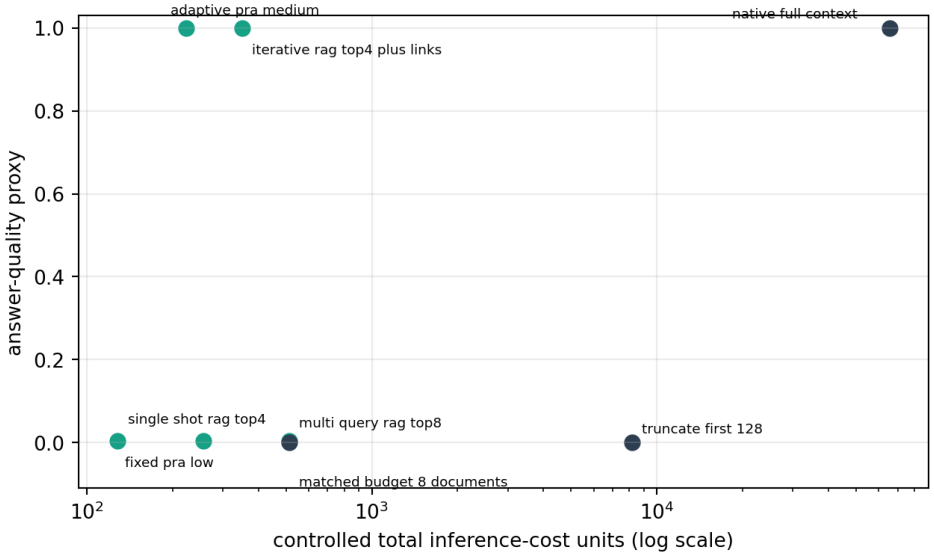

Figure 6: Controlled two-hop quality/cost frontier. Iterative RAG is the appropriate comparison for iterative PRA; a one-shot RAG baseline is insufficient.

Table 6: Controlled matched-corpus retrieval results. Quality is target-evidence recovery.

Method	quality	active K/V	cost units
one-shot RAG Top-4	.005	256	256
multi-query RAG Top-8	.005	512	512
iterative RAG Top-4 + links	1.000	320	352
fixed PRA low	.005	128	128
adaptive PRA medium	1.000	192	224
native full context	1.000	65,536	65,536

The corpus is constructed so the first retrieval identifies a bridge and the answer lies behind its link. One-shot methods fail by design; iterative RAG and adaptive PRA recover the target. PRA uses fewer controlled active tokens because it activates native memory rather than serializing all selected documents into a prompt. This supports the accounting distinction, not a general claim that PRA dominates optimized iterative RAG.

7.2 K/V selection proxies

At budget 32, the H2O, SnapKV, StreamingLLM, ClusterKV, and native-Q/K proxies recover the planted salient token with rates .915, 1.000, .120, .045, and 1.000. At budget 128 the rates are 1.000, 1.000, .240, .255, and 1.000. These outcomes reflect the controlled score construction. They test matched-budget bookkeeping and expose the expected failure of recent-only selection when evidence is not recent. They do not rank the upstream systems on real language generation.

Dense and external retrieval solve different physical problems. H2O and SnapKV select or compress an existing sequence cache; RAG retrieves text for prompt ingestion; PRA selects addressable contextual K/V. Longformer [2], BigBird [16], Reformer [4], and Routing Transformer [12] change the attention architecture and require compatible checkpoints. Compressive [11], memorizing [14], Infini-Attention [10], Landmark Attention [9], RETRO [3], and Titans [1] use recurrent, compressed, or external memory. We preserve this taxonomy rather than forcing unfair experimental rows.

8 SDK and Reimplementation Contract

The runtime is exposed through Python data classes and the `pra-hf adaptive` CLI. A caller can inspect profiles or resolve an automatic plan:

```
pra-hf adaptive profiles --output adaptive_effort_profiles.json
pra-hf adaptive plan --mode auto --features request_features.json \
  --controller controller.json --max-retries 2 \
  --max-search-budget 8 --max-active-kv 1024 \
  --latency-budget 1.5 --confidence-threshold 0.35 \
  --trace-output request_plan.json
```

Manual mode uses the same profile representation and accepts explicit facet, root, neighbor, hop, conceptual-budget, K/V, threshold, layer, granularity, and materialization overrides. Overrides are request-local; they do not silently redefine the validated global ladder.

An independent implementation needs four invariants:

1. Separate conceptual and physical budgets in the public request and trace schema.

2. Reject evaluator labels before feature vectorization; do not merely promise not to use them.

3. Preserve prior selected identities and pages when escalating, and account for regenerated tokens separately.

4. Label every timing as component, end-to-end, measured, or extrapolated. A Python microbenchmark cannot become a production throughput claim by omission.

The relevant implementation is compact. `adaptive_runtime.py` defines profiles, feature validation, learned and hand controllers, stop policy, retries, consistency, and calibration. `serving_runtime.py` defines native-Q/K indexes, merged interval gather, page storage, eviction, and ragged packing. The experiment scripts freeze validation decisions and emit every normalized row needed to regenerate the figures and tables.

9 Discussion

9.1 What adaptive control accomplished

The strongest result is not that adaptive control always saves compute. It is that the controller distinguishes a heterogeneous cohort from a uniformly hard one. QASPER contains many E0-sufficient cases, and the learned policy removes 60% of mean effort at matched chain quality. HotpotQA does not offer that opportunity under the present router, so the controller spends E2. A controller that reported large savings there would likely be sacrificing quality or exploiting labels.

The retry result is similarly conditional. Retry recovers failures, but confidence must stop it early enough to repay regeneration. Direct minimum-effort prediction is more valuable on QASPER; cheap-first retry is a robust fallback. Better answer-stability signals, incremental decoding, and prefix-cache reuse are the next systems targets.

9.2 Cognitive interpretation

The tested computational principle is modest:

$$\text{cheap or predicted effort} \rightarrow \text{uncertainty detection} \rightarrow \text{selective escalation} \rightarrow \text{stop}.$$

Natural cognitive systems plausibly vary retrieval cues and deliberation under uncertainty, but the present controller is not a biological model. Its scientific content is the measured frontier between quality and expected resource use.

9.3 Serving implications

Paper 3 converted conceptual selection into smaller active K/V. Paper 3.5 shows the control and accounting needed to exploit that bounded memory across requests. Exact GEMM should remain the small-index search default. ANN becomes relevant only beyond a measured crossover. Page layout is necessary for reuse and concurrency, but Python page lookup erases the expected gain. The next credible systems milestone is a fused GPU path that maps selected logical intervals directly to PagedAttention blocks and measures HBM, TTFT, TPOT, throughput, and eviction under concurrency.

10 Limitations

The adaptive quality endpoint is evidence-chain completion from frozen traces, not a live generated answer. Output confidence comes from a separate controlled model, so a combined routing/output

controller remains untested. The held-out adaptive cohort contains only 65 example–seed units and inherits Paper-2.5's model and dataset choices.

The learned controller is linear. It establishes a deterministic baseline but does not exploit temporal retry histories or richer question semantics. The output calibration cohort is small and imbalanced. Semantic consistency uses token F1. It can miss paraphrase and contradiction.

The page cache is a standalone implementation, not vLLM PagedAttention. Search, gather, caching, and batching timings are CPU component benchmarks. HBM concurrency values are extrapolated. FAISS, HNSW, Triton, CUDA extensions, and production cache libraries remain to be integrated. The coarse index underperforms exact GEMM at the tested scale.

RAG and K/V comparisons are controlled proxies. A complete external study must run strong single-shot and iterative RAG, upstream H2O/SnapKV/ClusterKV/StreamingLLM implementations, native long context, and matched generation on the same backbone and corpus. Sparse and recurrent-memory architectures require compatible checkpoints. Dollar-per-million-token estimates are unavailable on the local hardware.

Finally, no full PRA backbone is trained here. Learned consumer adaptation and full-weight scaling are deferred to Paper 4.

11 Conclusion

PRA can be represented as an adaptive bounded-memory runtime rather than one fixed retrieval configuration. A small validation-frozen controller preserves high-effort QASPER chain quality at 60% lower mean effort and correctly declines to save effort on predominantly hard HotpotQA. Monotonic retry recovers failures but is not free; generation overhead can outweigh reused search. Entropy alone is not a sufficient stop signal, while combined observable routing features provide useful but imperfect failure discrimination.

The systems prototype establishes exact batched search, K/V gather parity, page and cache accounting, and heterogeneous batching. It also identifies the unfinished work plainly: coarse search must beat GEMM at scale, page gathering must move out of Python, and adaptive control must run inside a concurrent production scheduler. The contribution is a reproducible control contract and an honest quality/cost frontier on which that next integration can be built.

A Artifact Manifest

All normalized artifacts are under `docs/papers/shared/results/paper3_5_adaptive_pra`. They include:

- effort profiles, controller features/training/calibration, oracle ceiling, regret, retry traces/outcomes, and adaptive frontier;
- indexed search, gather, page, cache, batching, serving, RAG, K/V, and long-context rows;
- baseline taxonomy, findings JSON, and all paper figures in PNG and PDF form.

The complete study runs with:

```
python -m experiments.paper3_5_adaptive_pra.run_study
python -m pytest
```

B Decision and Accounting Details

The learned controller uses standardized ridge least squares against one-hot E0/E1/E2 targets. The failure controller uses the same estimator with success/failure targets. Validation selects a .3 maximum predicted-failure threshold for retry stopping. AUROC is computed by exact pairwise ranking, AUPRC by average precision over descending risk, ECE with ten equal-width bins, and Brier score as mean squared probability error.

Risk–coverage sorts examples from lowest to highest predicted failure. At coverage c, selective risk is the failure fraction among the lowest-risk cN observations. We retain every point in the calibration JSON rather than reading values back from a rendered figure.

Search comparisons count query–key dot products. K/V bytes count both keys and values. Page fragmentation is allocated page slots minus valid tokens. Batch allocation counts padded or page rounded token slots. CPU benchmark bytes are physical tensor sizes; HBM/request is an explicitly marked extrapolation from token count, dimension, dtype, and K/V multiplicity.

C Representative Public Trace


```
{
  "attempt": 1,
  "effort": "E1_medium",
  "control_vector": {
    "F": {"policy": "multi_span", "count": 2},
    "R": 2, "K": 4, "H": 1,
    "B": {"conceptual_parents": 4, "native_kv_tokens": 512},
    "theta": 0.45,
    "L": {"search": [24, 27], "consumer": [26, 27]},
    "G": 128, "M": "evidence_centered"
  },
  "active_native_kv": 512,
  "incorrect_probability": 0.28,
  "retry_consistency": 0.83,
  "reused_search_items": 1,
  "reused_kv_tokens": 256,
  "stop_reason": "confidence"
}
```


This trace is a runtime decision record. It gives an application enough information to display effort and cost or request a manual retry, without exposing hidden reasoning.

References

[1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

[2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[4] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[8] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.

[9] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*, 2023.

[10] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.

[11] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[12] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: `https://arxiv.org/abs/2003.05997`.

[13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037, 2023.

[14] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

[16] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

[17] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: `https://openreview.net/forum?id=RkRrPp7GKO`.